

SmartRoom Manager

Réservation intelligente des salles

Dylan Menga Wanda · Angédarelle Ngangne Takoudjou · Maeva Nguadjong

ECE Paris — B3 · Sujet n° 2

github.com/DD542/SmartRoom-Manager

Réserver une salle, aujourd'hui

La scène que tout le monde a vécue

- On demande. On attend.
- On se déplace pour vérifier.
- Deux personnes, un créneau, un conflit.
- La salle réservée reste vide.

[VISUEL – photo ou pictogramme : une porte de salle avec un post-it « réservé ? », plein cadre, sobre]

Ce que le sujet demandait

Le périmètre attendu

- Portail avec calendrier
- Recherche multicritère
- Recommandation de salles
- Statistiques d'occupation

Compétences évaluées : web, contraintes, interfaces, bases relationnelles

Notre réponse

Une promesse, tenue par la base de données

- Le conflit n'est pas détecté.
- Il est rendu impossible.
- Et il débouche sur une alternative.

[SCHÉMA – trois blocs alignés : « Deux demandes » → « PostgreSQL refuse » → « Alternative proposée ». Flèches en accent #4F8CFF]

Les règles vivent en base

Configurables, pas figées dans le code

- Durée : 30 minutes à 4 heures
- Anticipation : 15 minutes à 60 jours
- Quota : 10 réservations actives
- Libération après 10 minutes sans check-in
- Validation requise au-delà de 20 places

[CAPTURE – écran d'administration « Règles de réservation », sélecteur de portée visible à gauche]

Architecture

Quatre couches, une règle par couche

domain/	règles pures – ni base, ni HTTP
models/	46 entités et leurs contraintes
services/	ce que l'application fait
api/v1/	160 opérations exposées

React · FastAPI · PostgreSQL 16 · Docker

- Le domaine ne connaît pas le framework.

La double réservation devient impossible

```
ALTER TABLE bookings ADD CONSTRAINT ex_bookings_no_overlap
EXCLUDE USING gist (
    room_id WITH =,
    time_range WITH &&
)
WHERE (status <> 'annulee' AND deleted_at IS NULL);
```

Garantie au niveau base, pas applicative

Une couche testable sans base

Ce que l'isolation rend possible

- Six types de chevauchement couverts
- Changement d'heure, créneaux adjacents
- 1 026 tests API, 455 tests interface
- Chaque correction porte son test

```
[CAPTURE – sortie de pytest : la ligne « 1026 passed », lisible, fond sombre]
```


Recommander, et l'expliquer

Un score pondéré, jamais un verdict opaque

- Capacité, équipements, bâtiment, étage
- Occupation et historique du compte
- Le score ordonne, il n'élimine pas
- Chaque proposition dit pourquoi

[CAPTURE – carte de salle recommandée avec son score et sa justification en toutes lettres]

Démonstration

Deux utilisateurs, un créneau

- Réserver en trois gestes
- Le conflit — refusé par la base
- L'alternative — proposée, acceptée
- La trace côté administration

Écran rendu à l'application — 5 minutes

Ce que les tests ont rattrapé

Trois défauts qu'aucune erreur ne signalait

- Matrice de permissions : codes contre identifiants
- Journal d'audit : le diff jamais affiché
- Filtres de salles : trois sur cinq muets

Un champ absent d'une réponse JSON ne lève rien

Le projet en chiffres

48 tables	110 contraintes CHECK / EXCLUDE
161 index	dont 51 partiels
160 opérations	d'API en 15 domaines
52 écrans	33 utilisateur, 19 administration
1481 tests	1026 API, 455 interface

Mesuré sur le dépôt, non estimé

Limites assumées

Ce que nous n'avons pas fait

- Pas d'authentification ADFS de l'école
- Pas d'application mobile installable
- Pas de mesure de charge
- Assistant local, non confronté aux usages

Perspectives

La suite, si le projet continuait

- Campagne de charge sur la concurrence
- Notifications poussées sur téléphone
- Journal d'audit étendu aux utilisateurs
- Export iCal d'un agenda complet

Merci

Vos questions

github.com/DD542/SmartRoom-Manager

Dylan Menga Wanda · Angédarelle Ngangne Takoudjou · Maeva Nguadjong